iREL Recruitment Task 2026

Code-Mixed Pedagogical Flow Extractor

Project Report

Swayam Goyal

Roll No. 2024111012

12 March 2026

Contents

1 Problem Statement **2**

2 Videos Selected **2**

3 System Architecture **2**
3.1 Pipeline Modules . . . 4

4 Transcription Approaches **4**
4.1 Sarvam AI `saarika-v2` — Best Cloud Quality . . . 4
4.2 Whisper `large-v3` (Offline) — Best Offline Quality . . . 4
4.3 Abandoned Approaches . . . 5

5 Audio Pre-Processing **5**

6 Stuck-Token and Hallucination Fix **5**
6.1 Problem Description . . . 5
6.2 Three-Layer Defence . . . 6

7 ASR Spelling Correction (Step 2.5) **6**
7.1 Problem . . . 6
7.2 Solution — `pipeline/asr_correct.py` . . . 6

8 LLM Pipeline and Model Tiering **7**
8.1 Three-Tier Model Configuration . . . 7
8.2 Rate Limit Handling . . . 7

9 Output Structure and Data Representation **7**
9.1 Why a DAG? . . . 7
9.2 Output Files Per Video . . . 8
9.3 `final_output.json` Schema . . . 8

10 Results **9**
10.1 Video 1 — `HZm1QyEGV-4` (Kannada–English, Python) . . . 9
10.2 Video 2 — `n4orr02dEvw` (Telugu–English, Python) . . . 9
10.3 Video 3 — `N8ZLYHDp97I` (Gujarati–English, Finance) . . . 10
10.4 Video 4 — `Ptlnehv--9k` (Bengali–English, Biology) . . . 10
10.5 Video 5 — `YIVPNbFKFjk` (Punjabi–English, Physics) . . . 10

11 Approach Comparison: Sarvam AI vs. Whisper large-v3 **10**

12 Design Decisions **11**

13 Setup and Reproduction **11**
13.1 Requirements . . . 11
13.2 Install . . . 11
13.3 Run — Whisper Pipeline (Offline) . . . 11
13.4 Run — Sarvam Pipeline (Cloud) . . . 12

1. Problem Statement

The task was to build an NLP pipeline that:

- Takes raw video from **5 code-mixed educational YouTube videos**.
- Extracts **core technical concepts** taught in each lecture.
- Performs **linguistic standardisation** — mapping colloquial / transliterated Indic terms to standard English academic terminology.
- Establishes **prerequisite dependencies** between concepts based on the pedagogical flow of the lecturer.
- Produces **structured, machine-readable output** with a visual dependency graph.

The central challenge was heavy **code-mixing** (e.g. Bengali–English, Telugu–English, Punjabi–English) and the accuracy degradation this causes in standard ASR and NLP systems.

2. Videos Selected

#	Video ID	Topic	Language Mix	Dur.
1	`HZm1QyEGV-4`	Loop Control Statements in Python	Kannada–English	7m 41s
2	`n4orr02dEvw`	Python Lists — mutability, indexing, slicing	Telugu–English	13m 30s
3	`N8ZLYHDp97I`	Financial Statements & Depreciation	Gujarati–English	7m 36s
4	`Ptlnehv--9k`	Living World: Growth & Development	Bengali–English	9m 46s
5	`YIVPNbFKFjk`	Introduction to Physics & Natural Phenomena	Punjabi–English	8m 36s

Table 1: Five code-mixed educational videos processed by the pipeline.

The videos were chosen to span **five different Indian languages**, **four academic subjects** (Computer Science, Biology, Finance, Physics), and code-mixing ratios ranging from 6 % to 30 %.

3. System Architecture

The pipeline consists of eight sequential steps:

```
YouTube URL / Local File
       |
       v
Step 1  Audio Extraction
        yt-dlp (download) -> ffmpeg
        -ac 1 -ar 16000 -af loudnorm -> .wav
       |
       v
Step 2  Speech-to-Text
```

```
        [OFFLINE] Whisper large-v3  (faster-whisper, CUDA)
        [ONLINE]  Sarvam AI saarika-v2  (cloud API)
        Native-script initial prompt per language
        Gap-recovery pass for stuck-token regions
       |
       v
Step 2.5  ASR Spelling Correction  (NEW)
        gpt-4o-mini  |  batches of 25 segments
        Fixes native-script phonetic errors; timestamps untouched
       |
       v
Step 3  Language Detection & Code-Mix Analysis
        Unicode script analysis per segment
       |
       v
Step 4  Normalisation  (Phi-4)
        Code-mixed -> clean academic English
        [TOPIC_SHIFT] markers + term-mapping table
       |
       v
Step 5  Concept Extraction  (Phi-4)
        core / supporting / prerequisite concepts
        Analogy detection, explanation method tagged
       |
       v
Step 6  Prerequisite Mapping  (Phi-4)
        HARD / SOFT dependency edges
        Strength score 0-1  |  DAG validation
       |
       v
Steps 7-8  Graph & Output
        NetworkX DAG  |  vis-network interactive HTML
        Matplotlib PNG  |  JSON final_output
```

3.1 Pipeline Modules

File	Role
`pipeline/audio_extract.py`	Step 1 — yt-dlp + ffmpeg (loudnorm)
`pipeline/transcribe_faster_whisper.py`	Step 2 — Whisper large-v3, gap recovery
`pipeline/transcribe.py`	Step 2 (alt) — Sarvam AI transcription
`pipeline/asr_correct.py`	Step 2.5 — gpt-4o-mini spelling correction
`pipeline/language_detect.py`	Step 3 — Unicode script analysis
`pipeline/normalize.py`	Step 4 — Code-mix → English
`pipeline/concept_extract.py`	Step 5 — Concept extraction
`pipeline/prerequisite.py`	Step 6 — Prerequisite mapping
`pipeline/graph_builder.py`	Steps 7–8 — DAG, HTML, PNG, JSON
`pipeline/main.py`	Sarvam pipeline orchestrator
`pipeline_whisper/main.py`	Whisper pipeline orchestrator
`pipeline/config.py`	Shared config (3-tier LLM models)

Table 2: Pipeline source files and their responsibilities.

4. Transcription Approaches

4.1 Sarvam AI `saarika-v2` — Best Cloud Quality

Sarvam AI is an Indian AI company whose `saarika-v2` model is purpose-built for Indian code-mixed speech, trained on large-scale Indic language data.

Audio is downloaded, converted to 16 kHz mono WAV, and POSTed to `api.sarvam.ai/speech-to-text` in chunks.

Results:

- Cleanest, most accurate transcription across all five Indic languages.
- Native-script output with very low phonetic error rate.
- Minimal hallucination even on fast or heavily code-mixed speech.

Limitation: Requires a Sarvam API key and active internet connection.

4.2 Whisper `large-v3` (Offline) — Best Offline Quality

OpenAI's Whisper large-v3 is the largest open-weight multilingual ASR model. We run it locally via `faster-whisper` for GPU-accelerated inference on CUDA (float16).

Initial problems:

1. **Latin-alphabet phonetic output.** Whisper correctly identified the language (BN/TE/KN) but spelled words in English letters, e.g. `zip jagat` instead of , `kokbifazon` instead of .
2. **Stuck-token loops.** On ambiguous audio segments, Whisper locked onto a single word (e.g.) and repeated it 200+ times, causing the duplicate detector to drop up to 400 seconds of audio from a 585-second video.

Fixes are detailed in Sections 5 and 6.

4.3 Abandoned Approaches

- **Whisper `medium`:** Higher word error rate and frequent language confusion on Indic content. Replaced by `large-v3`.
- **Sarvam Caption API:** Auto-generated YouTube captions on Indian channels are phonetically transcribed in English. Produced too much noise to be useful; switched to audio-based transcription.

5. Audio Pre-Processing

Raw downloaded audio has inconsistent loudness: speakers move closer to or further from the microphone, intro/outro music varies in volume, and recording quality differs across channels. This directly degrades Whisper accuracy.

We added the **EBU R128 loudness normalisation** filter to the ffmpeg command used exclusively by the Whisper pipeline:

```
ffmpeg -i input.mp3 \
  -ac 1 \            # mono
  -ar 16000 \        # 16 kHz (Whisper's native sample rate)
  -af loudnorm \     # EBU R128 normalisation -> -23 LUFS
  output.wav
```

This makes the integrated loudness uniform throughout the file, reducing both word error rate and hallucination at quiet/loud transitions.

Code change: `pipeline/audio_extract.py` — `extract_audio()` gained a `loudnorm: bool` parameter. The Sarvam pipeline passes `loudnorm=False`; the Whisper pipeline passes `loudnorm=True`.

6. Stuck-Token and Hallucination Fix

6.1 Problem Description

Whisper large-v3 with `condition_on_previous_text=True` can enter a feedback loop in code-mixed Indic audio:

1. Whisper encounters an ambiguous segment and outputs a wrong word with low confidence.
2. That wrong word is fed as context to the next 30-second window.
3. The next window, primed with wrong context, outputs the same wrong word again.
4. The cascade continues; the duplicate detector (`MAX_DUP_RUN=2`) correctly blocks the looping output, but drops coverage of all remaining audio.

6.2 Three-Layer Defence

Layer 1 — `condition_on_previous_text=False`. Each 30-second window is decoded from scratch using only the native-script `initial_prompt`. No poisoned context flows forward. This is the single most impactful fix.

Layer 2 — Tighter hallucination thresholds.

```
compression_ratio_threshold    = 1.5   # was 2.0
repetition_penalty             = 1.3
no_repeat_ngram_size           = 5
hallucination_silence_threshold = 2.0
```

Layer 3 — Gap detection + recovery re-transcription. After the main pass, the timeline is scanned for any uncovered region > 12 s. Each such gap is sliced from the raw audio array and independently re-transcribed:

```
gaps = find_uncovered_regions(segments, duration, threshold=12)
for start, end in gaps:
    gap_audio = audio_data[start*16000 : end*16000]
    recovered = model.transcribe(gap_audio,
                    condition_on_previous_text=False)
    segments += recovered
segments = sorted(segments, key=lambda s: s.start)
```

This guarantees that **no audio block exceeding 12 seconds is silently dropped**, regardless of model behaviour.

7. ASR Spelling Correction (Step 2.5)

7.1 Problem

Even with native-script prompts, Whisper occasionally produces phonetic spelling errors — words that *sound* right in the original language but are written incorrectly:

Whisper output	Correct form	Meaning
		characteristics
		different
		cell division

Table 3: Examples of phonetic spelling errors produced by Whisper for Bengali.

7.2 Solution — `pipeline/asr_correct.py`

A new Step 2.5 uses **gpt-4o-mini** to spell-correct raw segments before any downstream processing. The model receives a strict system instruction:

> *Fix phonetic spelling errors to correct native-script spellings. Do **not** translate anything. Do **not** add, remove, or reorder words. Keep all English words exactly as they appear.*

Segments are processed in **batches of 25** to stay within token limits. The corrected transcript overwrites the original `_transcript.json`. Timestamps are never modified — only text strings are corrected.

8. LLM Pipeline and Model Tiering

8.1 Three-Tier Model Configuration

Using a single LLM for every step is wasteful: ASR correction needs speed (many small calls), normalisation needs strong multilingual understanding, and concept/prerequisite extraction needs structured reasoning. We therefore split into three tiers:

Config variable	Default model	Step	Rationale
`LLM_MODEL_LIGHT`	`gpt-4o-mini`	2.5 — ASR correction	Fast; many small calls
`LLM_MODEL`	`Phi-4`	4 — Normalisation	Strong Indic multilingual; separate quota
`LLM_MODEL_REASON`	`Phi-4`	5–6 — Concepts & prereqs	Structured JSON reasoning

Table 4: Three-tier LLM model configuration across pipeline steps.

All tiers can be overridden via `.env`:

```
LLM_MODEL_LIGHT=gpt-4o-mini
LLM_MODEL=Meta-Llama-3.3-70B-Instruct
LLM_MODEL_REASON=DeepSeek-V3-0324
```

8.2 Rate Limit Handling

Improvements to `pipeline/llm_client.py`:

- Detects HTTP 429 / `rate limit` / `quota` errors.
- Normal errors: exponential backoff (2^n seconds).
- Rate-limit errors: $60 \times$ attempt seconds (60s, 120s, 180s, ...).
- `max_retries` raised from 3 to 5.

9. Output Structure and Data Representation

9.1 Why a DAG?

A prerequisite relationship is naturally a Directed Acyclic Graph: concept A must be understood before B, which is needed before C. Cycles are logically impossible in genuine pedagogical dependency. Enforcing DAG structure gives us:

- **Topological ordering** — the unique valid learning sequence.

- **Root concepts** — what the teacher assumes the student already knows.
- **Leaf concepts** — the terminal skills the lecture builds toward.
- **Longest path** — the critical chain of the lecture.

9.2 Output Files Per Video

For each `<VIDEO_ID>`, the pipeline writes to `data_whisper/<VIDEO_ID>/`:

File	Contents
`*_transcript.json`	Raw ASR output with per-segment timestamps and confidence
`*_normalized.json`	Clean English + `[TOPIC_SHIFT]` markers + term mapping
`*_concepts.json`	Concept list: name, type, description, analogy, key points
`*_prerequisites.json`	Dependency edges: from/to, type, strength 0–1, justification
`*_final_output.json`	Merged JSON: all above + graph metrics + lecture summary
`prerequisite_graph.html`	Interactive vis-network graph (hover, drag)
`prerequisite_graph.png`	Static Matplotlib graph for quick preview

Table 5: Output files written for each processed video.

9.3 `final_output.json` Schema

```
{
  "metadata": {
    "video_source": "https://youtube.com/watch?v=...",
    "transcription_model": "large-v3",
    "detected_language": "kn",
    "languages_detected": ["English", "Kannada"],
    "code_mix_ratio": 0.093,
    "audio_duration_seconds": 461.4
  },
  "lecture_info": {
    "topic": "Loop Control Statements in Python",
    "summary": "...",
    "teaching_flow": [...],
    "analogies_used": [...]
  },
  "concepts": [...],
  "prerequisites": {
    "dependencies": [
      {
        "from_concept": "concept_1",
        "to_concept": "concept_3",
        "relationship_type": "HARD_PREREQUISITE",
        "strength": 0.9,
        "justification": "..."
      }
    ],
```

```
    "learning_path": ["concept_1", "concept_3", ...],
    "root_concepts": ["concept_1"],
    "leaf_concepts": ["concept_7"]
  },
  "graph_metrics": {
    "total_concepts": 12,
    "total_dependencies": 6,
    "is_dag": true,
    "graph_density": 0.045,
    "longest_path_length": 5,
    "topological_order": [...]
  },
  "linguistic_analysis": {
    "technical_terms_mapped": { ... },
    "topic_shifts": [...]
  }
}
```


10. Results

Video ID	Language	Dur. (s)	Concepts	Deps	Terms mapped
`HZm1QyEGV-4`	Kannada–English	461	12	6	63
`n4orr02dEvw`	Telugu–English	810	10	9	97
`N8ZLYHDp97I`	Gujarati–English	456	6	5	9
`Ptlnehv--9k`	Bengali–English	586	8	7	41
`YIVPNbFKFjk`	Punjabi–English	516	8	7	282

Table 6: Summary metrics for all five processed videos. All graphs are valid DAGs.

10.1 Video 1 — `HZm1QyEGV-4` (Kannada–English, Python)

Topic: Loop Control Statements (`break`, `continue`, `pass`)
Code-mix: 9.3 % **12 concepts, 6 dependencies**
Learning path: `for loop` → `break` → `conditional statements` → `loop termination` → `continue` → `pass`

The dependency graph correctly placed `for loop` as the root, matching the lecturer's actual teaching order.

10.2 Video 2 — `n4orr02dEvw` (Telugu–English, Python)

Topic: Python Lists — mutability, indexing, slicing, operations
Code-mix: 23.3 % **10 concepts, 9 dependencies**
Learning path: `list` → `mutability` → `indexing` → `slicing` → `list operations` → ⋯ → `list comprehension`

Highest term-mapping count (97) driven by heavy Telugu code-mixing. The 9-edge DAG faithfully reconstructs the lecturer's bottom-up strategy.

10.3 Video 3 — `N8ZLYHDp97I` (Gujarati–English, Finance)

Topic: Financial Statements & Depreciation Calculation
Code-mix: 5.8 % (lowest) **6 concepts, 5 dependencies**
Learning path: `financial statements` → `sales calculation` → `depreciation` → `depreciation ledger` → `profit/loss` → `balance sheet`

Gujarati is underrepresented in most ASR training data; the native-script prompt approach directly mitigated its lower out-of-the-box accuracy.

10.4 Video 4 — `Ptlnehv--9k` (Bengali–English, Biology)

Topic: Characteristics of the Living World: Growth & Development
Code-mix: 12.3 % **8 concepts, 7 dependencies**
Learning path: `growth` → `cell division` → `differentiation` → `reproduction` → `DNA replication` → `metabolism` → `homeostasis` → `adaptation`

This was the most challenging video: Bengali phonetic errors were severe (e.g. →) and the stuck-token loop (word , at ≈ 278 s) dropped 300+ seconds of audio. Step 2.5 and the gap-recovery pass were both designed and validated on this video.

10.5 Video 5 — `YIVPNbFKFjk` (Punjabi–English, Physics)

Topic: Introduction to Physics & Natural Phenomena
Code-mix: 30.4 % (highest) **8 concepts, 7 dependencies**
282 terms were mapped to standard English — the largest mapping table in the dataset, reflecting the extreme code-mixing density. The normalisation step successfully converted dense Punjabi physics terminology (, ,) into academic English.

11. Approach Comparison: Sarvam AI vs. Whisper large-v3

Dimension	**Sarvam AI** `saarika-v2`	**Whisper** `large-v3`
Transcription accuracy	Best	Very good
Native-script output	Always correct	Correct after prompt fix
Code-mix handling	Purpose-built for Indian	Good with prompting
Internet required	Yes (cloud API)	No (fully offline)
API key required	Yes (Sarvam account)	No
Speed	Depends on API	≈ 0.2× realtime (GPU)
Cost	Pay-per-minute	GPU compute only
Phonetic errors	Low	Moderate (fixed by Step 2.5)
Stuck-token loops	Not observed	Occurs; fixed by gap recovery
Best for	Production / highest quality	Offline / privacy / no cost

Table 7: Head-to-head comparison of the two transcription backends.

Verdict: Sarvam AI produces better raw transcripts for Indian code-mixed content. The fully offline Whisper large-v3 pipeline — with loudnorm, native prompts, `condition_on_previous_text=F` gap recovery, and ASR correction — approaches Sarvam quality on downstream concept extraction.

12. Design Decisions

Why not Whisper's `translate` task? The `task="translate"` mode always outputs English, destroying the linguistic evidence needed to identify code-mixing patterns. We use `task="transcribe"` and handle translation in Step 4 (normalisation) with full topic context.

Why chunk normalisation? A 10-minute lecture produces 4,000–8,000+ tokens of transcript text. Sending everything in one LLM call produces degraded output. Chunking at $\approx$ 3,000 characters lets the model focus on local context.

Why validate DAG structure? If the model hallucinates a cycle (A requires B, B requires A), that is a detectable quality signal. All five videos produce valid DAGs; this validation catches prompt or model failures early.

Why two output directories (`data/` vs. `data_whisper/`)? Separating directories allows side-by-side comparison of Sarvam vs. Whisper outputs for the same video using identical skip flags (`--skip-audio`, `--skip-transcribe`, etc.) without overwriting data.

Why Phi-4 for normalisation? Microsoft Phi-4 (14B parameters) has strong multilingual coverage and excels at structured-output tasks. It also uses a separate API quota pool from the gpt-4o family, so rate limits on one tier do not block others.

13. Setup and Reproduction

13.1 Requirements

- Python 3.11+
- CUDA GPU (Whisper large-v3 at reasonable speed; CPU works at $\approx 10\times$ slower)
- `ffmpeg` (`sudo apt install ffmpeg`)
- `yt-dlp` (`pip install yt-dlp`)

13.2 Install

```
git clone <repo>
cd irel_task
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env   # add GITHUB_TOKEN and SARVAM_API_KEY
```

13.3 Run — Whisper Pipeline (Offline)

```
# Full run on a new video
python -m pipeline_whisper.main "https://www.youtube.com/watch?v=VIDEO_ID"

# Resume from a specific step (skip already-completed stages)
python -m pipeline_whisper.main "URL" --skip-audio
python -m pipeline_whisper.main "URL" --skip-audio --skip-transcribe
python -m pipeline_whisper.main "URL" --skip-audio --skip-transcribe
    --skip-correct
```

```
python -m pipeline_whisper.main "URL" --skip-audio --skip-transcribe
    --skip-correct --skip-normalize
```

13.4 Run — Sarvam Pipeline (Cloud)

```
python -m pipeline.main "https://www.youtube.com/watch?v=VIDEO_ID"
```